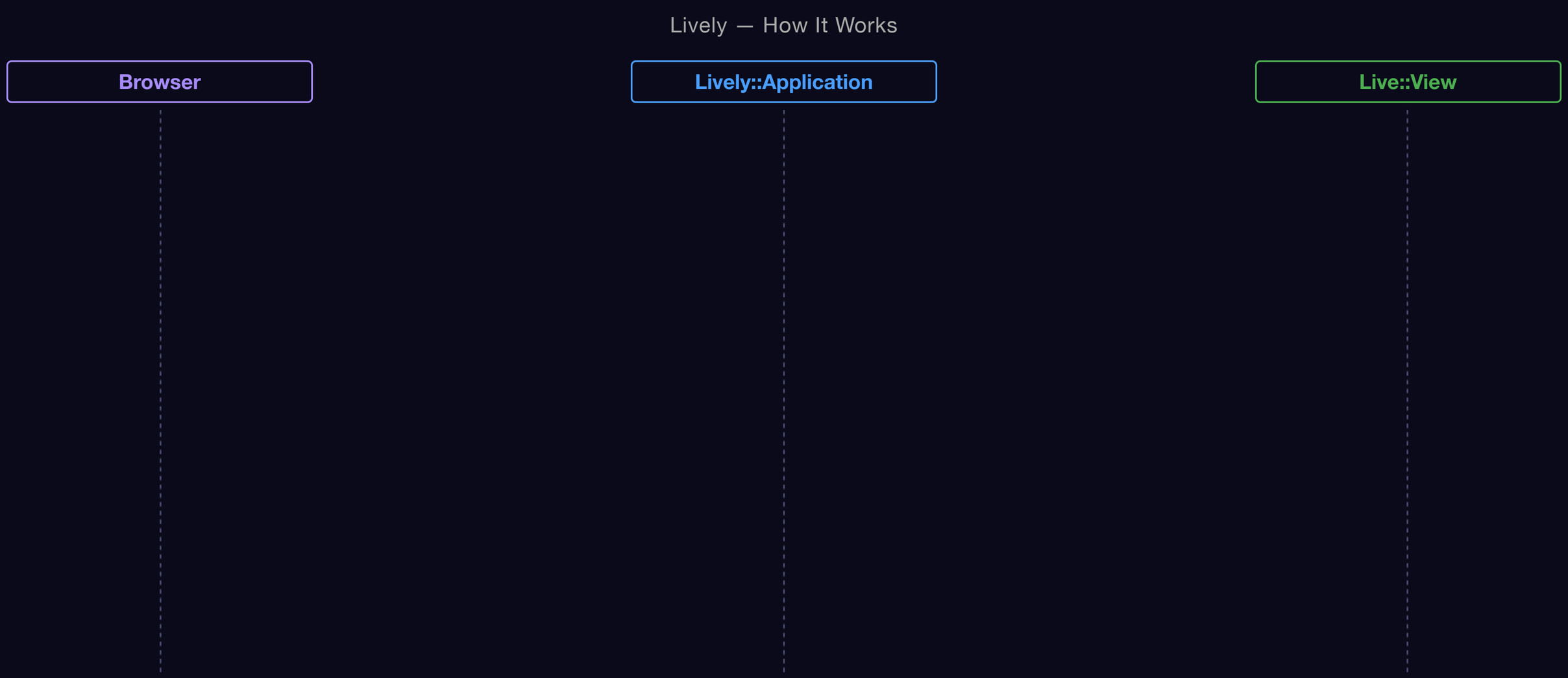
Lively

Real-time Ruby web applications, from a single file.

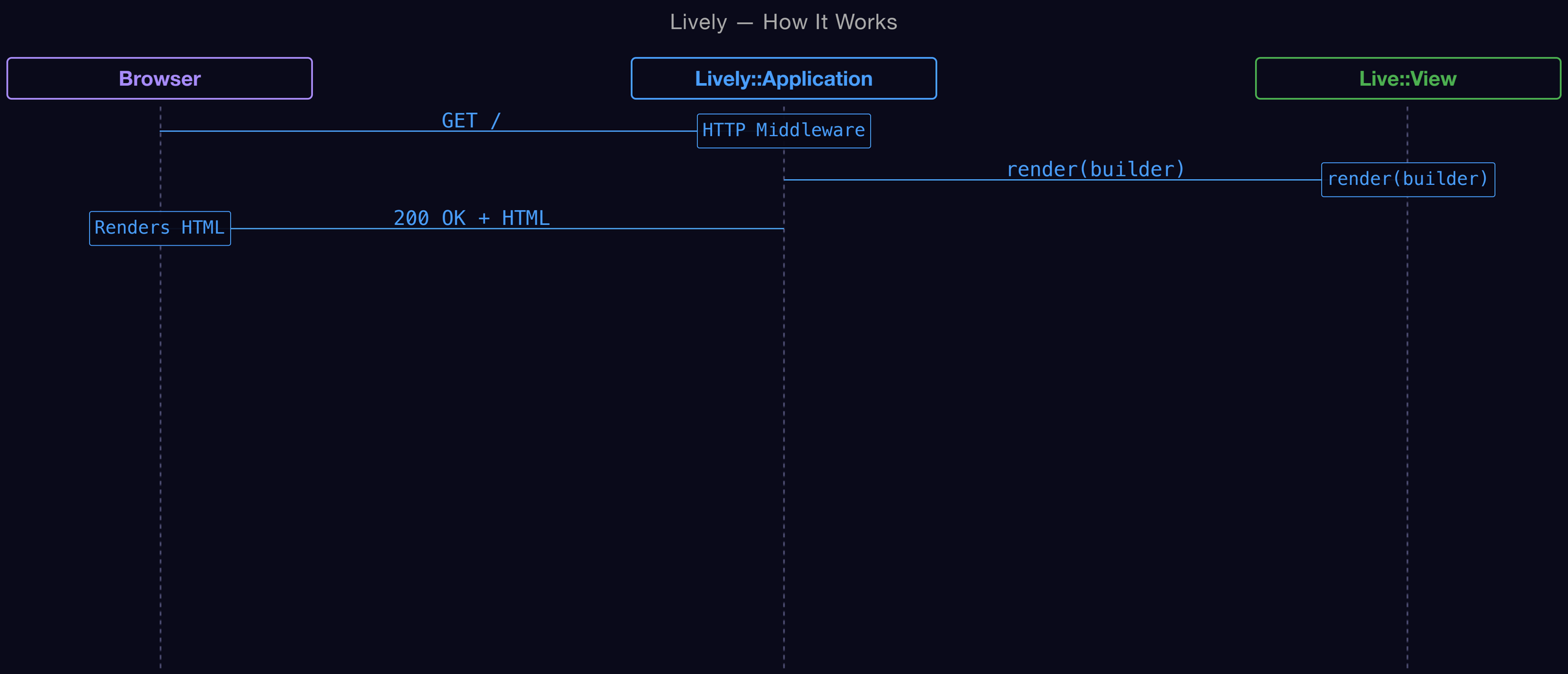
Welcome. Today I want to show you Lively — a Ruby framework for building interactive, real-time web applications. It's designed for creative coding: the kind of work where you want to experiment freely, see results immediately, and stay in Ruby the whole time.

Build interactive web apps in Ruby — no JavaScript required.

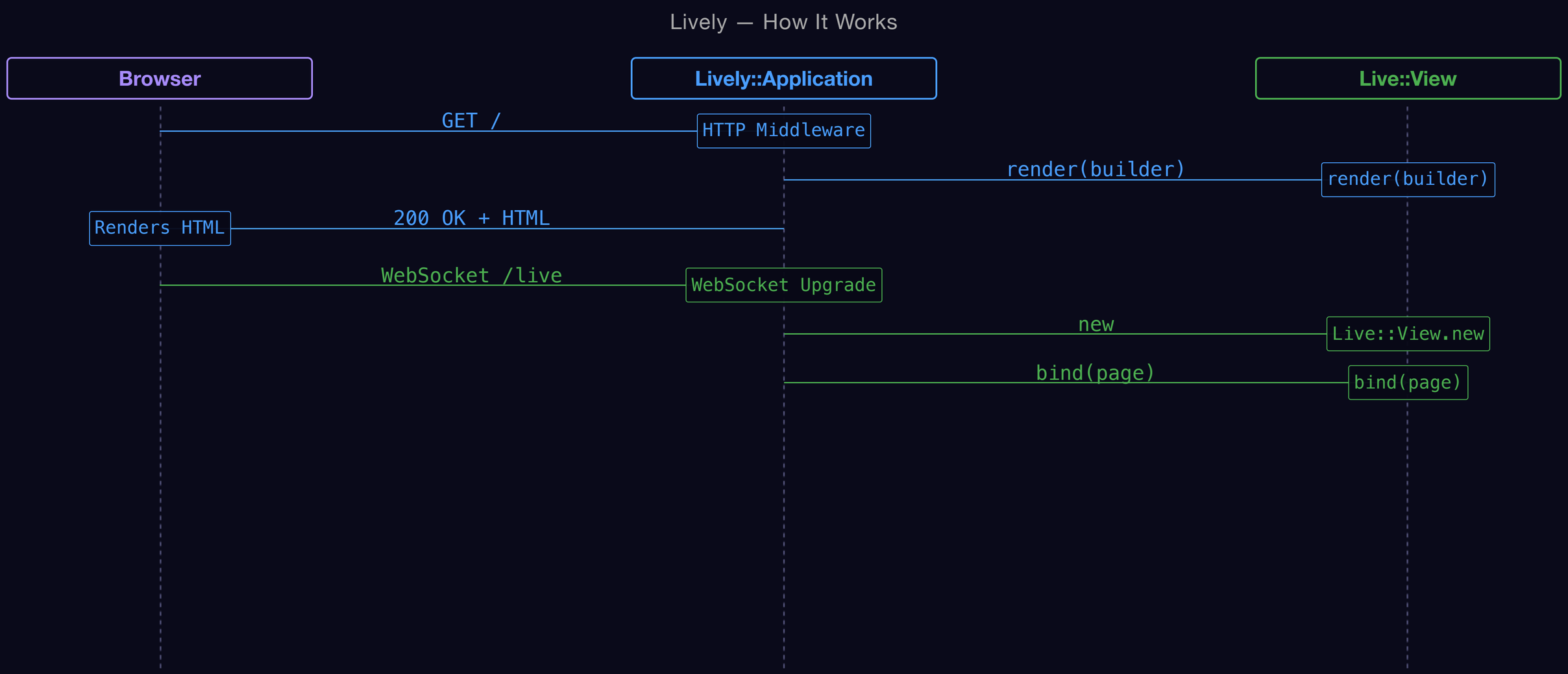
Lively lets you write server-side Ruby that pushes HTML updates to the browser in real time over a WebSocket. You define views, the server runs your logic, and the browser just renders what it receives.



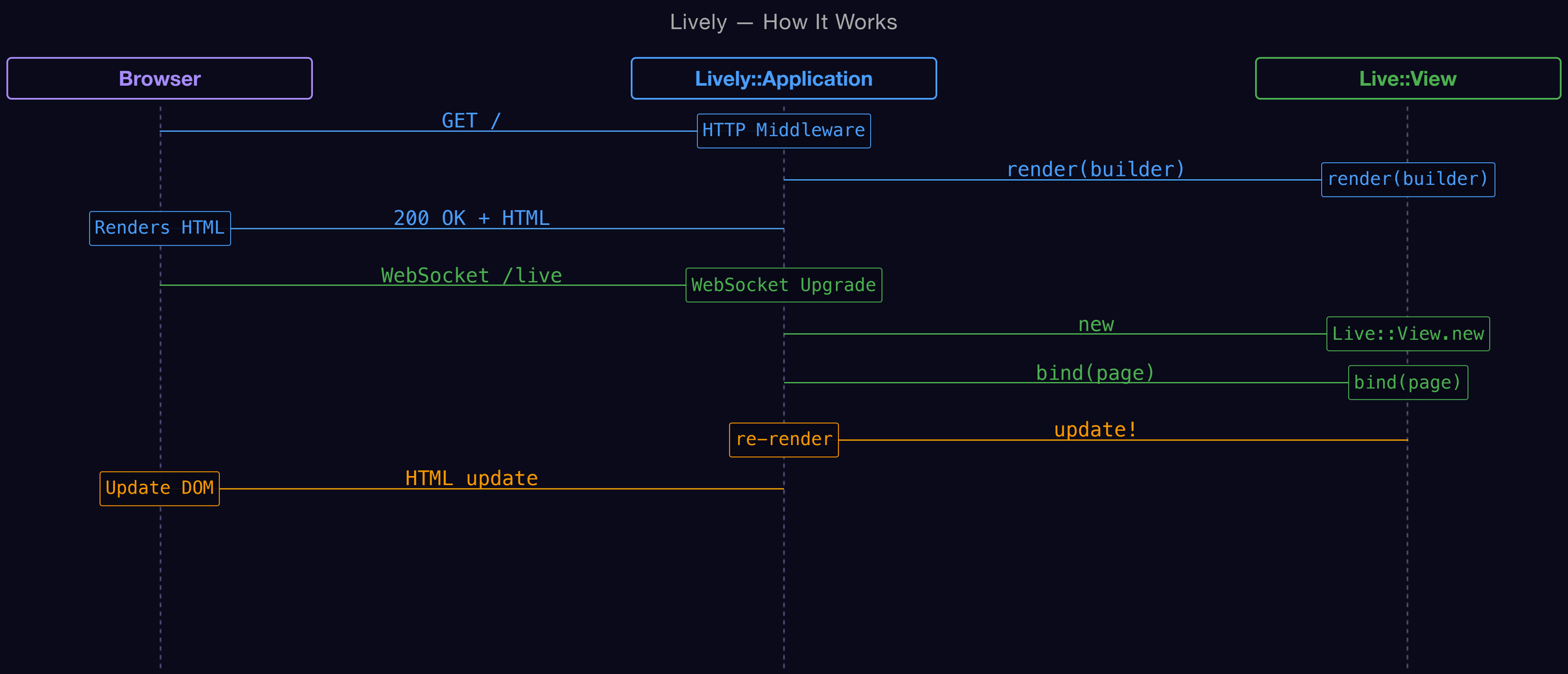
Every Lively application is a conversation between three actors: the Browser, the Application, and your View. Each has a clear role, and they communicate in a specific sequence.



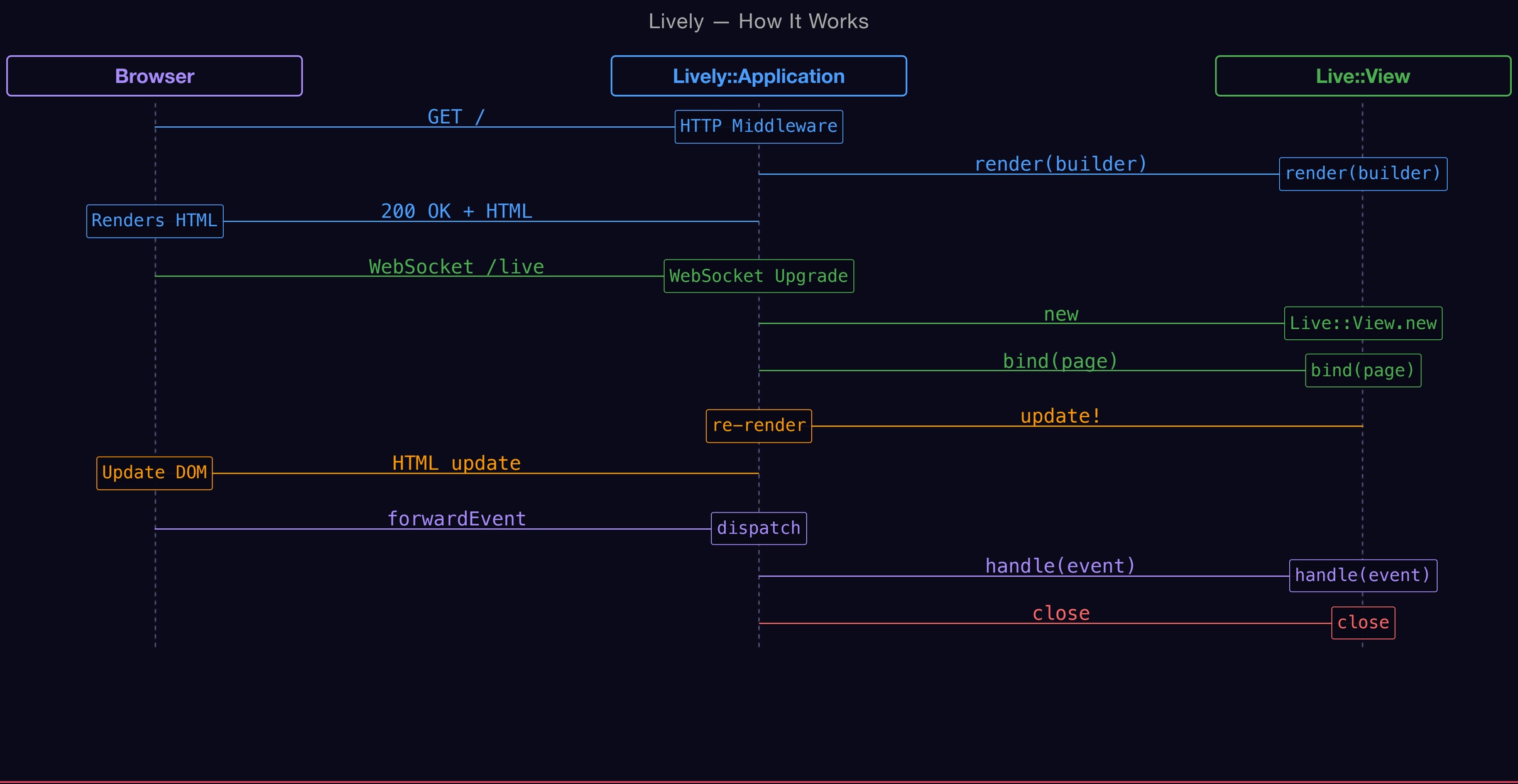
The browser sends a `GET` request. The Application asks your View to `render`, collects the HTML, and returns a normal `200 OK`. Nothing live yet — just a regular page load.



After the page loads, the browser opens a persistent WebSocket to `/live`. The Application instantiates a new View, then calls `bind` — your signal to start background tasks, subscriptions, or timers.



When something changes, call `update!`. The Application re-renders the View and pushes the updated HTML to the browser over the WebSocket, compressed with deflate — no full-page reload.



User interactions travel over the WebSocket and land in `handle`. When the tab closes or the user navigates away, `close` is called — the right place to cancel timers, unsubscribe, or release anything you started in `bind`.

Now let's look at how that's organised in code — the five classes you'll encounter in every Lively application.

- `Lively::Application` — Protocol::HTTP middleware; serves the page and handles WebSocket upgrades.

`Lively::Application` is a `Protocol::HTTP` middleware. It serves the initial page over HTTP, then upgrades the connection to a WebSocket for everything live.

- **`Lively::Application`** — Protocol::HTTP middleware; serves the page and handles WebSocket upgrades.
- **`Live::View`** — your component; defines `render` and handles events.

`Live::View` is your component. Define `render(builder)` to describe the HTML, and optionally `bind`, `handle`, and `close` to manage the full lifecycle of a connected client.

- **`Lively::Application`** — Protocol::HTTP middleware; serves the page and handles WebSocket upgrades.
- **`Live::View`** — your component; defines `render` and handles events.
- **`Live::Page`** — manages all views connected to one WebSocket session.

`Live::Page` ties multiple views to a single WebSocket session. Different parts of the screen can update independently — each view manages its own state and rendering.

- `Lively::Application` — Protocol::HTTP middleware; serves the page and handles WebSocket upgrades.
- `Live::View` — your component; defines `render` and handles events.
- `Live::Page` — manages all views connected to one WebSocket session.
- `Lively::Resolver` — reconnects views by class name after a page reload.

`Lively::Resolver` acts as the controller layer — it maps a class name to a view when a WebSocket connects. It also provides security: only explicitly registered classes can be instantiated, so arbitrary code can't be invoked from the client.

- **`Lively::Application`** — Protocol::HTTP middleware; serves the page and handles WebSocket upgrades.
- **`Live::View`** — your component; defines `render` and handles events.
- **`Live::Page`** — manages all views connected to one WebSocket session.
- **`Lively::Resolver`** — reconnects views by class name after a page reload.
- **`Lively::Assets`** — serves static files (CSS, JS, images) from `public/`.

`Lively::Assets` serves files straight from `public/` — CSS, JavaScript, images, fonts. Drop something in the folder and it's available immediately, no configuration required.

Hello World

Let's write the simplest possible Lively application.

application.rb

```ruby
#!/usr/bin/env lively

class HelloWorldView < Live::View
  def initialize(...)
    super
  end

  def bind(page)
    super
    self.update!
  end

  def render(builder)
    builder.tag(:p) do
      builder.text("Hello, World!")
    end
  end
end

Application = Lively::Application[HelloWorldView]
```


The whole application in one file. The shebang means you can run it directly. Let's look at each part.

application.rb

```ruby
#!/usr/bin/env lively

class HelloWorldView < Live::View
  def initialize(...)
    super
  end

  def bind(page)
    super
    self.update!
  end

  def render(builder)
    builder.tag(:p) do
      builder.text("Hello, World!")
    end
  end
end

Application = Lively::Application[HelloWorldView]
```


`initialize` is called when the view object is created — once per page load. Call `super` to let `Live::View` set up its internals. This is where you'd set instance variables for any state the view needs to hold.

application.rb

```ruby
#!/usr/bin/env lively

class HelloWorldView < Live::View
  def initialize(...)
    super
  end

  def bind(page)
    super
    self.update!
  end

  def render(builder)
    builder.tag(:p) do
      builder.text("Hello, World!")
    end
  end
end

Application = Lively::Application[HelloWorldView]
```


`bind` is called once the WebSocket connects and the view is attached to a live page. This is your signal to start any background tasks or subscriptions. Calling `update!` here triggers the first render and pushes the HTML to the browser.

application.rb

```ruby
class HelloWorldView < Live::View
  def initialize(...)
    super
  end

  def bind(page)
    super
    self.update!
  end

  def render(builder)
    builder.tag(:p) do
      builder.text("Hello, World!")
    end
  end
end

Application = Lively::Application[HelloWorldView]
```


`render` is called every time the view needs to update the browser. The `builder` argument is a tag builder — call `builder.tag` to emit HTML elements and `builder.text` to emit text nodes. Whatever you build here is sent to the browser over the WebSocket — compressed with deflate — and rendered in place, with no full page reload.

application.rb

```ruby
  def bind(page)
    super
    self.update!
  end

  def render(builder)
    builder.tag(:p) do
      builder.text("Hello, World!")
    end
  end
end

Application = Lively::Application[HelloWorldView]
```


`Lively::Application[HelloWorldView]` wires your view into the framework. It returns an application class that knows which view to serve and how to resolve WebSocket reconnections. Assign it to `Application` and Lively picks it up automatically.

Make it executable and run it:

```
$ chmod +x application.rb
$ bundle exec lively ./application.rb
```

Or with live reloading on save:

```
$ bundle exec io-watch . -- lively ./application.rb
```

Open http://localhost:9292 in your browser.

That's it. Lively uses Falcon under the hood, so it starts a server on port 9292. Add `io-watch` and every time you save, the server restarts and the browser reconnects automatically — tight feedback loop for live coding.

Real-Time Clock

Now let's make something that actually updates — a clock that ticks every second.

Adding a tick loop

```ruby
class ClockView < Live::View
  def bind(page)
    super

    @task = Async do
      loop do
        self.update!
        sleep 1
      end
    end
  end

  def close
    @task&.stop
    super
  end

  def render(builder)
    builder.tag(:h1) do
      builder.text("Hello, World!")
    end
  end
end
```


Focus on lines 4–12.

The key addition is an Async task inside `bind`. Async gives us lightweight fibers — this loop runs concurrently with other connections without blocking. Every second it calls `update!`, which triggers a re-render and pushes the updated HTML to the browser over the WebSocket. Notice `close` — always stop your task when the view is done, or it will leak.

Showing the time

```
        sleep 1
      end
    end
  end

  def close
    @task&.stop
    super
  end

  def render(builder)
    builder.tag(:h1) do
      builder.text(Time.now.strftime("%H:%M:%S"))
    end
  end
end

Application = Lively::Application[ClockView]
```


Focus on lines 18–22.

All we changed in `render` is the text — from a static string to `Time.now`. Because `render` is called fresh on every `update!`, the time is always current. The browser receives the updated HTML over the WebSocket and re-renders. The full clock ticks live in the browser.

Build With an Agent

Let's go from zero to a working application using an AI agent.

gems.rb

```
source "https://rubygems.org"

gem "lively"
gem "io-watch"
gem "agent-context"
```


Create a `gems.rb` with these three gems. `lively` is the framework, `io-watch` enables live-reloading during development, and `agent-context` is the key ingredient — it installs API documentation from your installed gems so your agent understands the libraries you're using.

bundle update

```
$ bundle update
Fetching gem metadata from https://rubygems.org/..........
Resolving dependencies...
Installing async 2.39.0
Installing async-websocket 0.30.0
Installing live 0.18.2
Installing agent-context 0.3.0
Installing falcon 0.55.3
Installing lively 0.17.1
Bundle updated!
```


`bundle update` pulls in everything Lively needs — the async ecosystem, Falcon, the Live runtime. The full dependency tree is about 30 gems, but you only declared three.

bundle exec bake agent:context:install

```
$ bundle exec bake agent:context:install
Installed context from 20 gems:
  async
  async-http
  async-service
  protocol-http
  protocol-websocket
  falcon
  lively
  ...
```


This scans every installed gem for bundled context files and copies them into `.context/` in your project. Your agent can now read authoritative documentation for the entire stack — no guessing, no hallucination.

"*Create a platformer game using Lively.*"

That's it. With the context installed, the agent can read the Lively guides and write a complete `application.rb` — views, state, game loop, keyboard handling — and get it right first time. You run it with `lively application.rb`. No boilerplate, no scaffolding, just Ruby.

"*Create a multiplayer asteroids game using Lively. Use a canvas for rendering.*"

Push it further. Multiplayer means shared state across multiple connected views — each player's inputs flowing into a single game loop, the canvas redrawn for everyone on every tick. The agent figures out the architecture, the synchronisation, and the rendering. You just describe what you want.

- **Worms** — multiplayer snake game with shared game state
- **Flappy Bird** — physics-based platformer in the browser
- **Game of Life** — cellular automaton ticking at 10fps
- **Chatbot** — streaming AI responses word by word
- **Adventure** — text adventure with server-side game logic
- **Platformer** — side-scrolling game with keyboard controls

All in the `examples/` directory. All in pure Ruby.

These aren't demos of what's theoretically possible — they're all in the repo. Each one is a single `application.rb` file. The framework handles the plumbing; you write the logic.

Thank You!

https://github.com/socketry/lively

Lively is open source under the MIT license — you can find it on GitHub under socketry. If you build something with it, I'd love to see it. Happy to take any questions.